Retained context and repeatable incident scenarios

Updated 21 October 2026. Product: TriageNorth; runtime package: `intelligent-operations`.

The retained-context contract

An alert is a small signal. Its surrounding logs and metrics live separately from the delivery ledger. Three rolling banks accept bounded normalized observations for each registered source under the current person's existing platform authority. The native implementation uses three logical SQL collections in the application's existing store; it does not provision three PostgreSQL servers or physical databases.

Each source has an explicit native resource kind and ID/scope attribute mapping, byte ceiling and record ceiling. Defaults start at 64 KiB / 64 records and permit bounded growth to 2 MiB / 2,000 records per bank. Rotation advances A → B → C → A when the active bank is full. Reusing a populated bank is legal only after its **newest receipt** is at least 900 seconds old. The clock is ingestion time, so a late observation cannot cause fresh accepted context to be erased early. All inserts, retry checks, rotation and counters commit together. A reused event ID with changed content is a conflict; an identical retry consumes no additional space.

If the next bank still contains protected context, the whole incoming batch receives HTTP 429 and rolls back. The producer must retain/retry it. Rotation is not permission to discard data. Three five-minute buckets alone would retain some observations for only ten minutes; that shortcut does not meet this contract. Bank capacity is currently budgeted. New streams can preserve and grow a protected bank up to the configured byte/record ceiling instead of erasing it. Growth never adds a database server or changes a physical PVC. At the ceiling, the whole batch is refused. Allow at least `peak bytes/second × 450 seconds` per bank for a steady stream, plus batch/serialization and workload headroom. Physical storage/index overhead is additional. Burst acceptance must be measured; finite capacity cannot guarantee unlimited ingestion. The operator can increase ceilings using the current configuration revision without changing source identity. Populated identity mappings and lower ceilings require a new source revision. Physical backing-store autoscaling remains separate work.

The minimum is not a promise of exact deletion at 15 minutes: a quiet bank can persist until safe reuse. Queries select the 15 minutes preceding the alert's event time, with an exact resource kind/scope/native ID and a maximum 200 returned observations. Missing or truncated context is explicit. This is an observed-data window, not a guarantee that the source collected every moment or that clocks are synchronized.

When filtering creates a capture reservation, the evidence consumer can pin that exact alert's context into a separate immutable packet. Pinning locks the stream against rotation. The packet retains source/hash/time provenance, bounded metric facts and log excerpts; it grants neither raw stream access nor execution authority. It survives raw-bank reuse. The initial profile joins the alert's own source revision only; joining external streams requires explicit cross-source evidence bindings. Basic credential redaction is an aid, not a guarantee that arbitrary log text contains no sensitive data. The packet remains untrusted evidence; log instructions never authorize tools.

Standard inbound formats

The following routes inherit the existing application role and caller scope:

Route, after `/api/intelligent-operations`	Accepted input
`POST /telemetry/configure`	Saved source ID, explicit mapping/limits, expected configuration revision.
`POST /streams/:sourceId/v1/logs`	Native OTLP/HTTP JSON `resourceLogs`; scalar attributes and string log bodies.
`POST /streams/:sourceId/v1/metrics`	OTLP/HTTP JSON numeric gauges and sums, units and explicit sum temporality.
`POST /streams/:sourceId/cloudwatch`	Decoded CloudWatch subscription JSON with exact account/group/stream mapping.
`POST /telemetry/window`	Saved source ID, exact resource identity and observation time.
`POST /telemetry/pin`	Durable capture-reservation ID, never a caller-supplied packet.
`GET /telemetry`	Current caller's mappings and three-bank counters.

Requests are at most 64 KiB of payload, 64 observations, 64 scalar attributes and 16 KiB per normalized observation. OTLP integer values that cannot be represented accurately are refused. Same-time observations retain distinct identities. Successful OTLP exports return an empty Export response. Unsupported histogram/summary/traces/protobuf/gRPC profiles are not silently converted. This is a bounded JSON profile, not full OTLP implementation certification. Anonymous or unmapped producers are not admitted.

Use the standard OpenTelemetry Collector for RFC 5424 syslog, file logs and cloud receivers. The application exposes no unauthenticated UDP listener. AWS subscription transport uses base64/gzip; decompression belongs to the trusted ingress/collector before the decoded endpoint, not to an inbound function-hosted adapter. A deployment must bind that producer through the existing platform connection/identity framework. Direct authenticated HTTP intake and a temporary real Collector export are accepted below. Unattended producer enrollment and permanent deployment remain separate acceptance gates.

Primary contracts: [OTLP](https://opentelemetry.io/docs/specs/otlp/), [RFC 5424](https://datatracker.ietf.org/doc/html/rfc5424), [CloudWatch subscriptions](https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/SubscriptionFilters.html), [CloudWatch Collector receiver](https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/receiver/awscloudwatchreceiver).

Dataset cache and provenance

`lab/datasets.json` pins upstream commits, file sizes and SHA-256 hashes. Run:

```sh
python3 intelligent-operations/lab/prepare_datasets.py
```

The cache is outside the application bundle, under `~/.cache/triagenorth/datasets`. Preparation verifies existing files as well as downloads. No benchmark quality claim follows from a successful download.

Dataset	Prepared material	Intended use and limit
Loghub BGL	2,000 sampled HPC log lines and structured parsing rows.	Alert/non-alert categories; sampled lines are not complete incident timelines.
Loghub OpenStack	2,000 sampled laboratory cloud logs and structured rows.	Compute/network/storage language; source includes injected failures, not live provider telemetry.
Loghub Linux	2,000 sampled server log lines.	Host/syslog parsing and noise controls; no incident ground truth assumed.
Loghub HDFS	2,000 sampled distributed-storage log lines.	Block/job identity parsing; no complete transaction sequence assumed.
NAB	EC2 latency/ASG CPU series and synthetic normal controls, original labels.	Time-separated anomaly evaluation; preserve duplicate timestamps and avoid future leakage.

The combined cache contains 18 files and 3,492,896 bytes. Loghub's notice permits research/academic use and requires attribution/notice retention. These samples are optional research inputs; they are not redistributed in the product. Consult [Loghub and its citations](https://github.com/logpai/loghub) and its retained LICENSE. Published [AWS VPC flow examples](https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs-records-examples.html) define flow formats; those examples are not a labeled datacenter-failure dataset.

Scenario lab: synthetic data through actual services

The new Scenario lab creates a separate `replay-*` environment under the signed-in operator. Its datacenter application host depends on a cloud database, which depends on a network interface. A second host has independent disk pressure. The test controls are intentionally synthetic and use no cloud credentials or infrastructure mutation.

The expected journey is **100 occurrences → 40 percolation matches → 4 surviving symptoms → 2 dependency groups → 4 pinned evidence packets**. Sixty routine observations are filtered; 36 repeat symptoms are suppressed. Four source-matched context observations are supplied using OTLP and CloudWatch-format messages. The UI shows exact paths, packet hashes, expected versus actual results and a partial-run error when appropriate. It does not substitute expected counts for service results. An actual OpenSearch and PostgreSQL contract verifies this cohort, including delivery retry behavior.

Next scenario gates include delivery storms, escalation/resolution, out-of-order clocks, missing CloudWatch identity, partial inventory, stale/changed dependency paths, disappearing hosts, insufficient context, source outage and conflicting cross-account resource names. Further recorded-data evaluations, statistical anomaly calibration, continuous collectors, automatic evidence-to-ticket dispatch and external ticket sync remain distinct work. Later, select read-only AWS/Azure/GCP educational environments with explicit identities, budgets and cleanup; no accounts or paid resources are created as part of dataset

preparation.

Source verification

All 83 Operations package tests pass, zero skipped, against disposable PostgreSQL 16 and OpenSearch 3.3.2. They include the actual Chromium Scenario lab at desktop/390-pixel phone widths, access-refusal clearing, four real pinned packets, independent SQL pools, full-batch rollback, the 900-second rotation boundary and server-verified saved receipts. Native installed acceptance and normal publication are recorded separately below.

Recorded baseline and installed acceptance

The genuine BGL evaluation preserves original epoch times, stable same-time LineId ordering and a virtual event-time suppression clock. Neither its Label field nor the expected outcome enters the source event or stored query. A level-derived severity baseline against actual OpenSearch gives: 2,000 lines; 403 matches; 1,597 filtered; 329 admitted symptoms; 74 suppressed repeats; 143 true positives; 260 false positives; 1,597 true negatives; no false negatives. Precision35.48%, recall100%, false-positive rate14.00%. This is a deliberately simple baseline over a sampled research corpus, not a trained anomaly detector or datacenter incident accuracy claim. The measured receipt is shipped at deploy/evidence/recorded-bgl.json and shown in Scenario lab.

Run the opt-in source benchmark only with the explicit disposable PostgreSQL35689 and OpenSearch35690 services described in engine/README.md:

```sh node intelligent-operations/lab/evaluate_recorded.cjs ```

On the installed native kernel, deploy/replay-acceptance.cjs signs in through the actual HTML form and runs the real Scenario lab:100→40→4→2 with four one-observation packets, complete dependency paths, exact hashes and desktop/phone layout checks. It passes on the installed application revision recorded in deploy/evidence/replay.json, including bank-growth/benchmark UI, batched replay, actual grouped cards and saved cohort reopening.

The accepted real-time canary uses lab/retention_verify.py start, waits at least902 real seconds while other work proceeds, then runs finish. It does not expose a clock override or edit stored timestamps. Its result must be recorded separately from the fast SQL clock fixture. PostgreSQL9-case telemetry tests now also prove bounded growth and reviewed capacity increases without identity changes or raw-data loss.

Submission artifacts and delivery

The eight-slide demo deck reads measured receipts rather than hard-coded success claims. The14-page editable Visio/PDF adds the retention/pinning contract; this guide also exports to Word/PDF. The normal full Store gate passes after its required Publishing runtime is supplied, with6partial and12explicit unavailable-prerequisite skips. Those checks are not certified as passed. Source checkpoints dbb862a0, 7a119bfc, a3d8d4a5 and 3c557c4a are published to the existing feature branch. The native acceptance receipt identifies the installed application revision; the kernel image has not changed.

Alert replay now uses a20-item bounded source-bound batch endpoint to avoid a separate bridge launch for every occurrence. Each delivery retains its independent transaction and retry identity. Mixed outcomes return207 and explicit per-item statuses; a source unavailable to the caller refuses the entire request before writes. The Scenario lab also renders actual grouped resource/dependency cards, with run-specific
```

cloud IDs so separate replays cannot collide. The final native receipt accepts this UX/batch path. No ticket, model run or infrastructure action is launched by replay.

The installed real-time canary passed after909.788seconds: early reuse returned429; BankA then advanced to generation1, leaving two original observations in the raw alert-time window while the immutable packet retained all three with an unchanged hash. No server clock or stored timestamp was changed. Native bounded-growth proof also retained12rows across three banks, refused the thirteenth at the configured ceiling, and accepted it after a reviewed ceiling increase without changing source identity. See deploy/evidence/retention.json and retention-growth.json.

Raw telemetry retry identity is bounded by the retained banks: an exact retry while its row is retained consumes no additional space. After safe raw reclamation, a very late replay may be accepted again; the permanent alert ledger and durable packet identities are separate. Raw retention is a minimum-age contract, not indefinite telemetry deduplication.

Actual standard Collector acceptance

lab/stream_verify.py now proves RFC5424/TCP → pinned OpenTelemetry Collector0.138.0 → OTLP/HTTP JSON → native rolling store, plus one genuine gauge sent through the Collector's OTLP receiver. Exactly two syslog observations and one numeric metric arrived under the configured host ID/site scope. The Collector was temporary, bounded to0.5CPU/256MiB, had no GPU or infrastructure credential, and was removed afterward. It used the operator's temporary password session over local loopback; no bootstrap token was copied. This accepts a real standard transport/encoding path, not permanent delegated producer enrollment. Compression is explicitly none because this native profile accepts JSON bodies. The pinned receiver requires a64KiB read buffer; the application's smaller observation/body limits still apply. Unsupported/oversized data needs an explicit producer transform/quarantine policy before production use.

The receipt is deploy/evidence/standard-stream.json. Source/transport setup must still use the shared connection and identity framework for unattended deployment, TLS, durable producer queues and expiring grants. Existing user-session acceptance is not that production enrollment.

Saved results and provenance

A completed scenario saves an immutable cohort in the existing current-user application store. The server accepts decision IDs and the correlation ID, then reloads the owned filter decisions, exact scan/graph revision and pinned packets. It computes the counts and graph itself; client-supplied totals and foreign-owner references are refused. The same receipt retries idempotently. Reopening Scenario lab shows the latest saved cohort without submitting new events. The saved result preserves its original times and source IDs; it does not present old observations as current infrastructure health.

`GET /replays` uses app-open authority; `POST /replays` requires existing records-write authority. No new database credential, infrastructure tool or broad grant is introduced. The response returns at most ten saved cohorts. Raw telemetry reclamation does not erase the separate packet or replay record. Native acceptance checks fresh-page reopen; source browser acceptance also asserts that reopen sends no POST requests.